CS 415 Lab 4 - Processes and the exec system call

Lab Overview:

For this lab, we will be learning how to create a controlled number of processes and use them to execute other programs.

Warning: This lab could potentially crash your VM forcing a restart. Do not test more than 7 processes until 100% certain.

Core Tasks:

1. Create a child process.
2. Call exec to execute another program.
3. Create multiple processes to execute more programs.
4. Use existing code to monitor the current running processes.

Task Details:

1. Create a process.
    a. Use **fork()** to create a child process.
2. Call exec to execute another program.
    a. Use **execvp()** to execute "./iobound -seconds 5".
    b. Wait for the child process to finish.
3. Create multiple processes to execute more programs.
    a. Ask the user from the command line for the number of processes (execute with ./lab.out n) where n is the number of processes.
    b. Create an array of pid_t
    c. Utilize a loop to start the exact number of processes.
    d. Wait for all the children processes to finish (using "**waitpid()**").
4. Open another terminal and use the command "top" to check the number of processes you created. Note, the processes' names are "./iobound"
5. Once you are certain that you can spawn the correct number of processes, you may use existing code to monitor the current running processes as you intended to start for the rest of this project.
    a. Use the following function in the code provided to generate the monitor script
        void script_print (pid_t pid_ary, int size)*
        NOTE: this function does not work on IX-Dev, you will receive the following error:

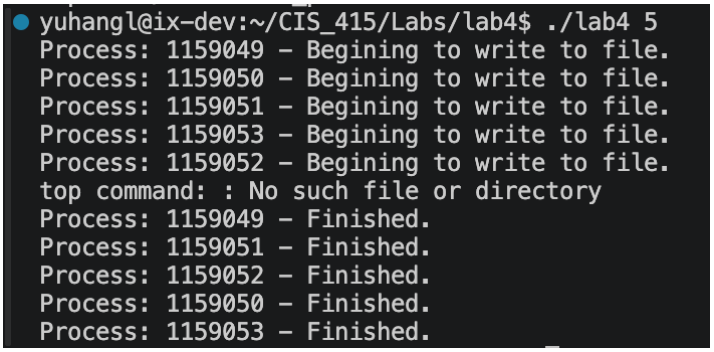
top command:: No such file or directory.

Ignore this error. Use step 4 (another terminal connected to IX-Dev) for monitoring processes.

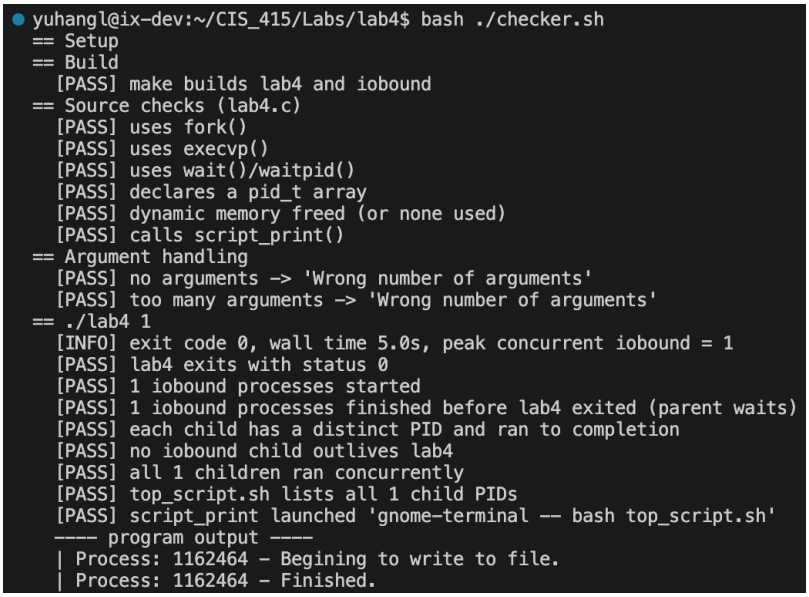
You should still include this function as described in your code for testing purposes.

NOTE: if you are running a VM then this function may or may not work depending on your setup. Do NOT do this step if you do not need to. If it does not work, try running "unset GTK_PATH" in a VS-code terminal.

6. The example output is shown below:



7. The example output of checker script is shown below:



```
== ./lab4 3
  [INFO] exit code 0, wall time 5.0s, peak concurrent iobound = 3
  [PASS] lab4 exits with status 0
  [PASS] 3 iobound processes started
  [PASS] 3 iobound processes finished before lab4 exited (parent waits)
  [PASS] each child has a distinct PID and ran to completion
  [PASS] no iobound child outlives lab4
  [PASS] all 3 children ran concurrently
  [PASS] all children started before any finished
  [PASS] top_script.sh lists all 3 child PIDs
  [PASS] script_print launched 'gnome-terminal -- bash top_script.sh'
  ---- program output ----
  | Process: 1162879 - Begining to write to file.
  | Process: 1162880 - Begining to write to file.
  | Process: 1162881 - Begining to write to file.
  | Process: 1162879 - Finished.
  | Process: 1162881 - Finished.
  | Process: 1162880 - Finished.
== ./lab4 5
  [INFO] exit code 0, wall time 5.1s, peak concurrent iobound = 5
  [PASS] lab4 exits with status 0
  [PASS] 5 iobound processes started
  [PASS] 5 iobound processes finished before lab4 exited (parent waits)
  [PASS] each child has a distinct PID and ran to completion
  [PASS] no iobound child outlives lab4
  [PASS] all 5 children ran concurrently
  [PASS] all children started before any finished
  [PASS] top_script.sh lists all 5 child PIDs
  [PASS] script_print launched 'gnome-terminal -- bash top_script.sh'
  ---- program output ----
  | Process: 1164074 - Begining to write to file.
  | Process: 1164078 - Begining to write to file.
  | Process: 1164073 - Begining to write to file.
  | Process: 1164075 - Begining to write to file.
  | Process: 1164077 - Begining to write to file.
  | Process: 1164074 - Finished.
  | Process: 1164078 - Finished.
  | Process: 1164073 - Finished.
  | Process: 1164075 - Finished.
  | Process: 1164077 - Finished.
== Valgrind (./lab4 2)
  [PASS] no definite leaks or memory errors

== Summary: 36 passed, 0 failed
```

Provided Files:

1. lab4.c
2. iobound.c
3. Makefile
4. checker.sh

Submission Requirements:

In order to receive points for this lab, the student must do the following:

1. (50% points) Attend the lab and make sure TA records your attendance.
2. (50% points) Complete this lab (refer to the following for submission instructions)

To make sure your work is graded correctly, submit a completed lab in zip/tar file that contains all source code files, Makefile, input/output_reference.txt, and the checker script.

The naming convention of the zip/tar file while uploading to Canvas

- FirstName_LastName_LabX (an example is given below)
- Name: John Doe

- Submission for: Lab4
- So the name of the zip/tar file should be:

  John_Doe_Lab4.tar **or** John_Doe_Lab4.zip